

RAG Automation API

A FastAPI Backend for Document Ingestion and Conversational
RAG

Technical Documentation

Isha Bhattarai

ishabhattacharai21@gmail.com

June 1, 2026

Abstract

RAG Automation API is a modular FastAPI backend for document ingestion and retrieval-augmented generation. It accepts PDF and TXT documents, extracts and chunks their text using a selectable strategy, generates Gemini embeddings, and stores vectors in Qdrant. The conversational API retrieves relevant context without using `RetrievalQAChain`, preserves multi-turn chat history in Redis, and uses an LLM to detect and persist interview bookings in PostgreSQL.

Contents

1	Overview	3
1.1	Technology stack	3
2	Architecture	3
2.1	Ingestion flow	3
2.2	Conversational RAG flow	4
3	Environment setup	4
3.1	Prerequisites	4
3.2	Installation	4
4	Module reference	5
4.1	Chunking strategies	5
4.2	Vector retrieval	6
4.3	Database schema	6
5	API reference	6
5.1	POST <code>/api/v1/ingest</code>	6
5.2	POST <code>/api/v1/chat</code>	6
5.3	Health endpoints	7
6	Testing	7
6.1	Health check	7
6.2	Ingestion with both strategies	7
6.3	Retrieval-grounded answer	7
6.4	Multi-turn memory	7
6.5	Interview booking	7

7	Design decisions	8
8	Conclusion	8

1 Overview

The project provides two REST APIs:

- **Document ingestion:** upload a `.pdf` or `.txt` file, extract text, select a chunking strategy, generate embeddings, store vectors, and save document metadata.
- **Conversational RAG:** ask document-grounded questions, retain Redis-backed conversation history, resolve follow-up questions, and store LLM-detected interview bookings.

The implementation does not use FAISS, Chroma, a user interface, or `RetrievalQAChain`. Retrieval and prompt construction are implemented directly in the application services.

1.1 Technology stack

Component	Technology	Purpose
API framework	FastAPI	Async REST endpoints
LLM	Gemini 2.5 Flash	Answers and booking extraction
Embeddings	Gemini embedding-001	768-dimensional vectors
Vector database	Qdrant	Similarity search
Session memory	Redis	Multi-turn chat history
Relational DB	PostgreSQL	Metadata and bookings
ORM and migrations	SQLAlchemy and Alembic	Persistence management
PDF parser	PyMuPDF	PDF text extraction
Tokeniser	NLTK Punkt	Sentence-based chunking

2 Architecture

2.1 Ingestion flow

1. The client uploads a PDF or TXT document and selects `fixed` or `sentence` chunking.
2. FastAPI validates the extension and extracts text.
3. The chunker splits extracted text with the selected strategy.
4. Gemini generates a vector for each chunk using the `retrieval_document` task type.
5. Qdrant stores vectors with document ID, filename, and chunk text.
6. PostgreSQL stores document metadata.

2.2 Conversational RAG flow

1. The client sends a `session_id` and query.
2. Redis loads the recent conversation history for that session.
3. Gemini checks the current query and history for interview-booking intent and the required fields.
4. If a complete booking is found, PostgreSQL stores it and the API returns a confirmation.
5. Otherwise, Gemini embeds the query using the `retrieval_query` task type.
6. Qdrant compares the query vector with stored chunk vectors using cosine similarity and returns the five most similar chunks.
7. The application builds a grounded prompt manually and Gemini generates the answer.
8. Redis stores the user message and assistant response.

3 Environment setup

3.1 Prerequisites

- Python 3.11 or higher
- Docker and Docker Compose
- A Gemini API key from [Google AI Studio](#)

3.2 Installation

```
git clone https://github.com/ishaa-bht/palm-mind-rag.git
cd palm-mind-rag

python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Create a `.env` file in the project root:

```
GEMINI_API_KEY=your_gemini_api_key_here
DATABASE_URL=postgresql+asyncpg://palmuser:palmpass@localhost:5433/palmdb
QDRANT_HOST=localhost
QDRANT_PORT=6333
QDRANT_COLLECTION_NAME=documents
REDIS_URL=redis://localhost:6379
APP_ENV=development
EMBEDDING_DIM=768
```

Start infrastructure, apply migrations, and run the API:

```
docker compose up -d
alembic upgrade head
uvicorn app.main:app --reload
```

Swagger UI is available at <http://localhost:8000/docs>.

4 Module reference

Module	Responsibility
<code>app/main.py</code>	Creates the FastAPI app, registers routers, and initializes the Qdrant collection during startup.
<code>app/api/ingest.py</code>	Implements PDF and TXT ingestion, extraction, chunking, embedding, vector upsert, and metadata persistence.
<code>app/api/chat.py</code>	Implements conversational retrieval, Redis memory, LLM answers, and booking persistence.
<code>app/services/chunker.py</code>	Provides fixed-size overlapping chunks and NLTK sentence-group chunks.
<code>app/services/embedder.py</code>	Generates document and query embeddings with Gemini.
<code>app/services/vector_store.py</code>	Creates the Qdrant collection, upserts embedded chunks, and retrieves the top five matches using cosine similarity.
<code>app/services/memory.py</code>	Stores up to 20 Redis messages per session with a one-hour inactivity TTL.
<code>app/services/llm.py</code>	Builds the custom grounded prompt and generates answers without <code>RetrievalQAChain</code> .
<code>app/services/booking.py</code>	Extracts name, email, date, and time from conversational booking intent.
<code>app/models/db_models.py</code>	Defines document metadata and booking SQLAlchemy models.

4.1 Chunking strategies

Strategy	Configuration	Recommended use
<code>fixed</code>	500 characters, 50-character overlap	Dense or irregular text
<code>sentence</code>	Up to 5 NLTK-tokenised sentences	Structured prose and FAQs

4.2 Vector retrieval

Qdrant stores one 768-dimensional Gemini embedding for each document chunk. During chat, the application embeds the user's query and performs a **cosine-similarity search**. Qdrant ranks stored chunk vectors by their directional similarity to the query vector, and the API sends the top five matching chunks to Gemini as grounded context.

Parameter	Value
Vector database	Qdrant
Distance metric	Cosine similarity
Embedding dimensions	768
Retrieved chunks per query	5

4.3 Database schema

Table	Stored fields
document_metadata	id, filename, file_type, chunking_strategy, chunk_count, uploaded_at
bookings	id, name, email, date, time, session_id, created_at

5 API reference

5.1 POST /api/v1/ingest

Uploads and processes a document using multipart form data.

Field	Type	Description
file	File	Required PDF or TXT file
strategy	String	Required: fixed or sentence

```
curl -X POST http://localhost:8000/api/v1/ingest \
-F "file=@test_doc.txt" \
-F "strategy=sentence"
```

5.2 POST /api/v1/chat

Answers questions using retrieved document context and Redis session history.

```
curl -X POST http://localhost:8000/api/v1/chat \
-H "Content-Type: application/json" \
-d '{"session_id":"user-001","query":"What skills does John have?"}'
```

5.3 Health endpoints

Endpoint	Response
GET /	API status and message
GET /health	{"status": "healthy"}

6 Testing

6.1 Health check

```
curl http://localhost:8000/health  
# {"status": "healthy"}
```

6.2 Ingestion with both strategies

```
curl -X POST http://localhost:8000/api/v1/ingest \  
-F "file=@test_doc.txt" -F "strategy=fixed"  
  
curl -X POST http://localhost:8000/api/v1/ingest \  
-F "file=@test_doc.txt" -F "strategy=sentence"
```

6.3 Retrieval-grounded answer

```
curl -X POST http://localhost:8000/api/v1/chat \  
-H "Content-Type: application/json" \  
-d '{"session_id": "test-001", "query": "What skills does John have?"}'
```

6.4 Multi-turn memory

Use the same `session_id` for a follow-up question:

```
curl -X POST http://localhost:8000/api/v1/chat \  
-H "Content-Type: application/json" \  
-d '{"session_id": "test-001", "query": "Where did he work?"}'
```

6.5 Interview booking

```
curl -X POST http://localhost:8000/api/v1/chat \  
-H "Content-Type: application/json" \  
-d '{"session_id": "booking-001", "query": "Book an interview for Jane Doe,  
jane@example.com, on 2026-06-10 at 14:00."}'
```

Inspect saved records from the PostgreSQL container:

```
docker compose exec postgres \
  psql -U palmuser -d palmdb \
  -c "SELECT * FROM bookings ORDER BY created_at DESC;"
```

7 Design decisions

- **Custom RAG pipeline:** retrieval, context formatting, and prompting are explicit application steps instead of a framework chain.
- **Qdrant vector store:** vectors and payload metadata are persisted in a dedicated similarity-search database. Cosine similarity is used to retrieve the five chunks most relevant to each query.
- **Redis session memory:** short-lived chat history supports follow-up questions while keeping the API stateless.
- **PostgreSQL persistence:** structured document and booking records are managed with SQLAlchemy and Alembic.
- **Selectable chunking:** callers can choose predictable character windows or sentence-aware grouping based on document shape.

8 Conclusion

RAG Automation API meets the core backend requirements for document ingestion, custom retrieval-grounded conversation, Redis-backed multi-turn memory, and LLM-assisted interview booking. Its modular service boundaries allow each integration to be tested and improved independently while keeping the public API compact.